

Analysis of algorithm

Measuring efficiency of an algorithm:

- Time: How long the algorithm takes (running time)
- Space: Memory requirement

Time and Space

- Time depends on processing speed
- Impossible to change for given hardware
- Space is a function of available memory
- Easier to reconfigure, augment
- Typically, we will focus on time, not space

Measuring running time:

- Analysis independent of underlying hardware
Measure in terms of “basic operations” Typical
- basic operations
 - ✓ Compare two values
 - ✓ Assign a value to a variable

Input size

- Running time depends on input size
- Larger arrays will take longer to sort
- Measure time efficiency as function of input size
 - Input size n
 - Running time $T(n)$
- Different inputs of size n may each take a different amount of time
- Typically, $T(n)$ is worst case estimate

Analysing the insertion sort problem

Case 1:- Worst case (When array is reverse order)

$T(n)$ = maximum time for any input of size 'n'

$$\begin{aligned} T(n) &= \sum_{\text{all statements}} (\text{cost of statement}) \cdot (\text{no. of times statement is executed}) \\ &= c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^n t_j + c_5 \sum_{j=2}^n (t_j-1) + \\ &\quad c_6 \sum_{j=2}^n (t_j-1) + c_7(n-1) \end{aligned}$$

Since $A[j] > \text{key}$ is always true.

$$\text{Therefore, } t_j = j \Rightarrow \sum_{j=2}^n t_j = \sum_{j=2}^n j = \sum_{j=1}^n j - 1 = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n j - 1 = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$$

$$\Rightarrow T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \frac{n(n+1)}{2} - 1 + c_5 \frac{n(n-1)}{2} + c_6 \frac{n(n-1)}{2} + c_7(n-1)$$

$$\Rightarrow T(n) = \left[\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2}\right]n^2 + \left[c_1 + c_2 + c_3 + c_7 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2}\right]n - [c_2 + c_3 + c_4 + c_7]$$

$\Rightarrow T(n)$ can be expressed as $an^2 + bn + c$

$\Rightarrow T(n)$ is quadratic in n .

Case 2:- Best case (When array is sorted)

since array is already sorted, therefore, all t'_j s are 1.

$$\begin{aligned}\Rightarrow T(n) &= c_1n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_7(n-1) \\ &= (c_1 + c_2 + c_3 + c_4 + c_7)n - (c_2 + c_3 + c_4 + c_7)\end{aligned}$$

$\Rightarrow T(n)$ as $an + b$, where a and b , are depends on the statement cost c_i

$\Rightarrow T(n)$ is a linear function of n .

Activity Selection Problem

P: n activities require exclusive use of a common resource, such as a lecture hall, Let $S = \{a_1, a_2, \dots, a_n\}$ be set of activities.

a_i needs resource during period $[s_i, f_i)$, a half-open interval, where, s_i is the start time and f_i is the finish time.

Assume that activities are sorted by finish time: $f_1 \leq f_2 \leq \dots \leq f_n$

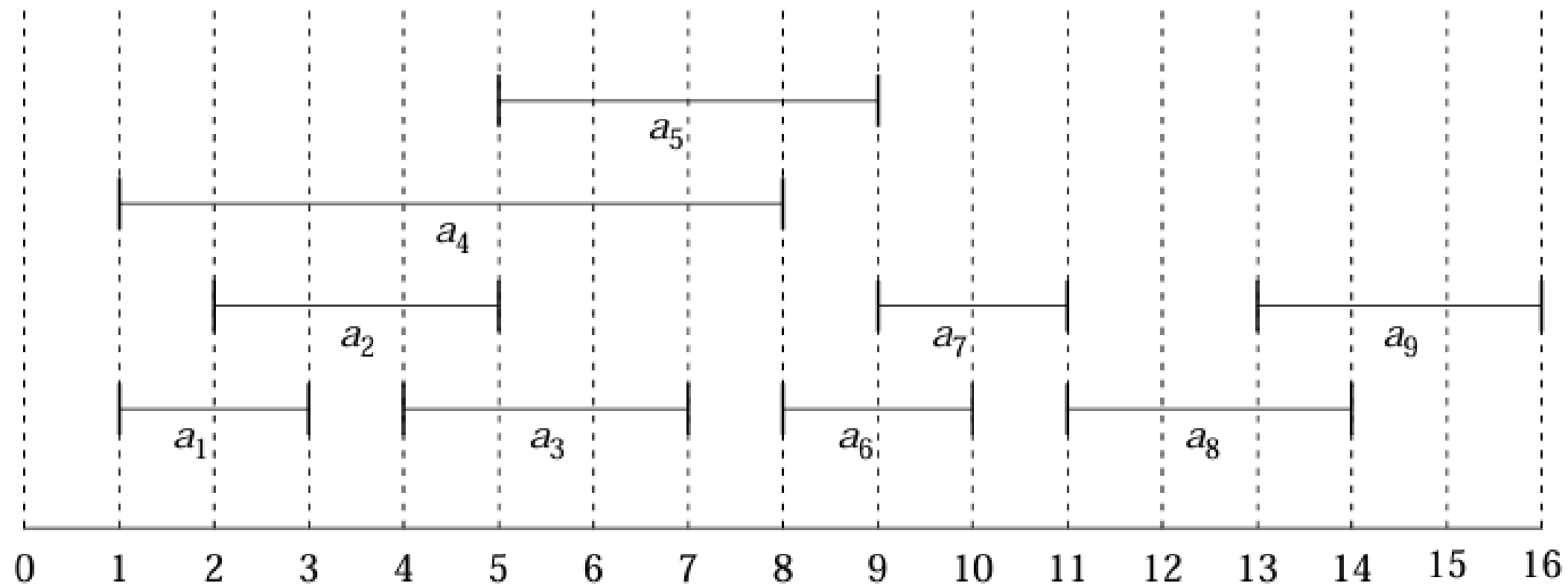
Objective: Select the largest possible set of nonoverlapping (mutually compatible) activities.

Here are a set of start and finish times:

i	1	2	3	4	5	6	7	8	9
s_i	1	2	4	1	5	8	9	11	13
f_i	3	5	7	8	9	10	11	14	16

Maximum-size mutually compatible set: $\{a_1, a_3, a_6, a_8\}$.

Not unique: also $\{a_2, a_5, a_7, a_9\}$.

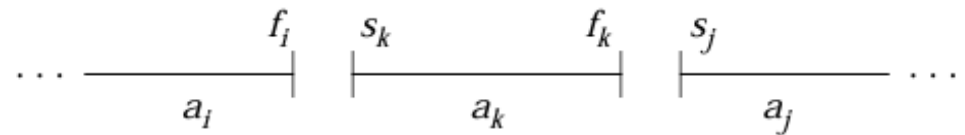


- **Sort activities** by their finishing times (earliest finish first).
- **Pick the first activity** (it finishes earliest).
- **Skip overlapping ones** — only choose the next activity that starts after the last chosen one finishes.
- **Repeat until done** — keep adding non-overlapping activities.

Optimal substructure of activity selection

$$S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$$

= activities that start after a_i finishes and finish before a_j starts .



Activities in S_{ij} are compatible with

- all activities that finish by f_i , and
- all activities that start no earlier than s_j .

Let A_{ij} be a maximum-size set of mutually compatible activities in S_{ij} .

Let $a_k \in A_{ij}$ be some activity in A_{ij} . Then we have two subproblems:

- Find mutually compatible activities in S_{ik} (activities that start after a_i finishes and that finish before a_k starts).
- Find mutually compatible activities in S_{kj} (activities that start after a_k finishes and that finish before a_j starts).

Let

$A_{ik} = A_{ij} \cap S_{ik}$ = activities in A_{ij} that finish before a_k starts ,

$A_{kj} = A_{ij} \cap S_{kj}$ = activities in A_{ij} that start after a_k finishes .

Then $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$

$\Rightarrow |A_{ij}| = |A_{ik}| + |A_{kj}| + 1.$

Claim

Optimal solution A_{ij} must include optimal solutions for the two subproblems for S_{ik} and S_{kj} .

Suppose we could find a set A'_{kj} of mutually compatible activities in S_{kj} , where $|A'_{kj}| > |A_{kj}|$. Then use A'_{kj} instead of A_{kj} when solving the subproblem for S_{ij} . Size of resulting set of mutually compatible activities would be $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A|$. Contradicts assumption that A_{ij} is optimal. ■ (claim)

Knapsack Problem

The **knapsack problem** is a classic optimization problem in computer science and operations research. It models the challenge of selecting items with given weights and values to maximize total value without exceeding a weight limit.

Types of Knapsack Problems:

1. 0-1 Knapsack

- Items are indivisible: take it or leave it.
- Decision variable: $x_i \in \{0,1\}$
- Solved using dynamic programming.
- Time complexity: $O(nW)$

2. Fractional Knapsack

- Items are divisible: take fractions.
- Decision variable: $0 \leq x_i \leq 1$
- Solved using greedy algorithm.
- Time complexity: $O(n \log n)$

Problem Statement:

A thief robbing a store finds n items. The i^{th} item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. The thief wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack, for some integer W . The goal is to choose items to maximize total value without exceeding W .

We have,

n items, each with:

- Value v_i
- Weight w_i

Knapsack capacity: W

Goal: maximize total value by possibly taking **fractions of items**.

Fractional Knapsack Strategy:

- Compute **value-to-weight ratio** for each item:

$$v_i/w_i$$

- Sort items in **descending order of ratio**.
- Take as much as possible of the highest-ratio item, then move to the next, until the knapsack is full.

Suppose we have items:

Item	Value (v_i)	Weight (w_i)	Ratio (v_i/w_i)
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Knapsack capacity $W = 50$.

Order: Item 1 (6.0), Item 2 (5.0), Item 3 (4.0).

- Take Item 1 fully: weight = 10, value = 60. Remaining capacity = 40.
- Take Item 2 fully: weight = 20, value = 100. Remaining capacity = 20.
- Take Item 3 partially: can take $20/30 = 2/3$ fraction.

$$\text{Value} = 120 \times \frac{20}{30} = 80$$

Total Value:

$$60 + 100 + 80 = 240$$

Pseudocode:

FRACTIONAL-KNAPSACK(v, w, W)

$load = 0$

$i = 1$

while $load < W$ and $i \leq n$

if $w_i \leq W - load$

 take all of item i

else take $(W - load)/w_i$ of item i

 add what was taken to $load$

$i = i + 1$

Huffman Coding

- **What is Huffman Coding Algorithm?**

Huffman Coding is a popular *greedy algorithm* invented by David A. Huffman in 1952, designed to compress data by generating optimal prefix codes. It assigns shorter codes to more frequent symbols and longer codes to less frequent ones, thereby reducing the overall bit-length required to encode data.

- The key characteristic is that the codes are prefix-free, meaning no code is a prefix of another, ensuring unambiguous decoding.

Constructing a Huffman code

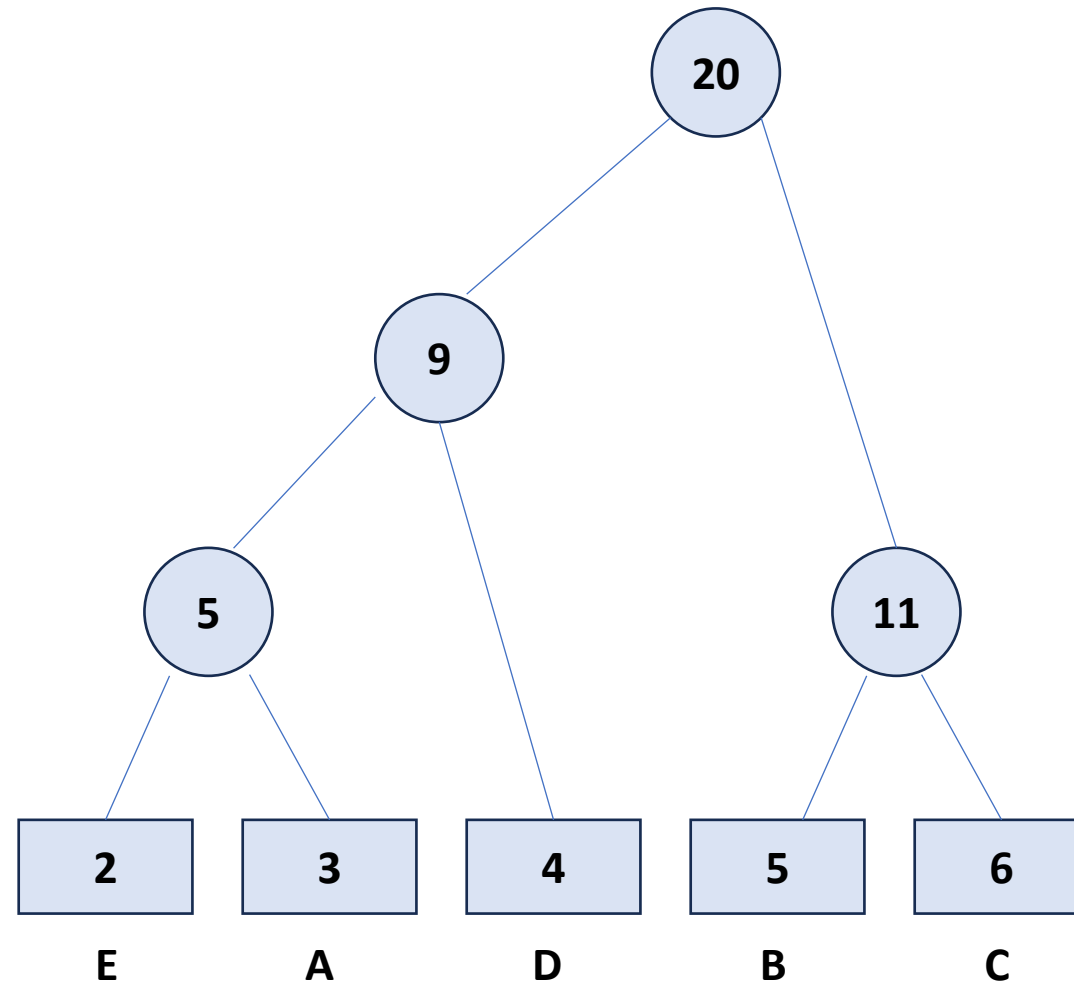
- **Step 1:** Construction of Huffman Tree
- **Step 2:** Traverse the Huffman Tree

Step 1: Construction of Huffman Tree

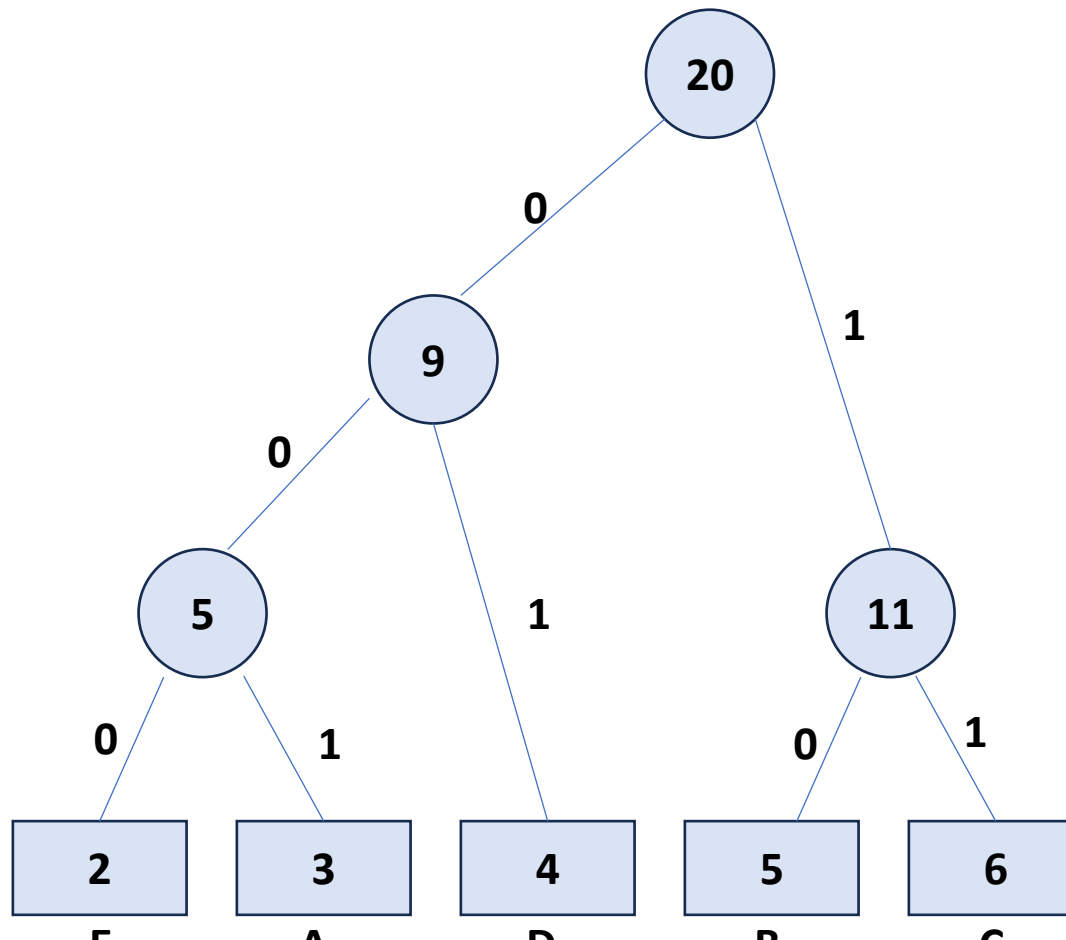
Consider: BCCABBDDAECCBBAEDDCC- 20 Characters

- Arrange the given characters in ascending order with respect to frequency/count

2	3	4	5	6
E	A	D	B	C



Step 2: Traverse the Huffman tree



0-left
1-right

Finding Huffman Code for each character

- Traverse the tree from the root node to reach the leaf nodes.

Character	Frequency/Count	Code
A	3	001
B	5	10
C	6	11
D	4	01
E	2	000

- $3*3 + 5*2 + 6*2 + 4*2 + 2*3 = 45$ bits
- From table: $5*8 + 12 = 52$ bits
- **Total = 45 + 52 = 97 bits**

- How the receiver decode the transmitted message??

10111100110100101001000111101000100001011111

DECODE

- Huffman codes compress data very effectively, savings of 20% to 90% are typical, depending on the characteristics of the data being compressed.
- Let us consider the Data to be sequence of characters.
- Huffman's greedy algorithm uses a table giving how often each character occurs (i.e., its frequency) to build up an optimal way of representing each character as a binary string.

- You have a **100,000-character file**, and only **6 characters** appear:
a, b, c, d, e, f with frequencies:

Character	Frequency (in thousands)
a	45
b	13
c	12
d	16
e	9
f	5

- **Fixed Length-coding**

- You need to assign a **unique binary codeword** to each character.
- Since there are 6 characters, you need at least:

$$6 \log_2 = 3 \text{ bits per character}$$

- a = 000
- b = 001
- c = 010
- d = 011
- e = 100
- f = 101

Total bits required:

$$100,000 \text{ characters} \times 3 \text{ bits} = 300,000 \text{ bits}$$

- **Huffman coding (variable-length)**

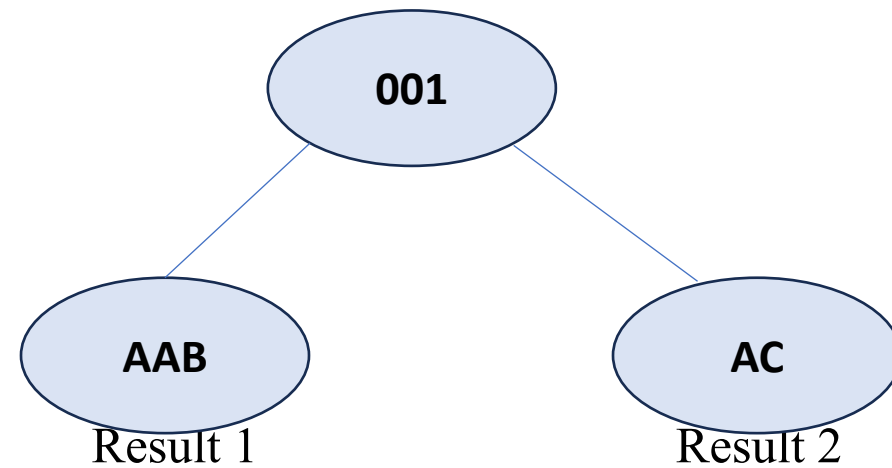
- Instead of using 3 bits for every character, Huffman coding assigns:
 - **Shorter codes to frequent characters** (like a)
 - **Longer codes to rare characters** (like f)

Character	Codeword
a	0
b	101
c	100
d	111
e	1101
f	1100

Total bits = $(45 * 1 + 13 * 3 + 12 * 3 + 16 * 3 + 9 * 4 + 5 * 4) * 1,000 = 224,000$ bits
a savings of approximately 25%.

- **Prefix Code**

- Huffman coding works on the technique called Prefix Code
- No Code is prefix of another code
- Eg:- a:0, b:1, c:01
- Message sent is 001

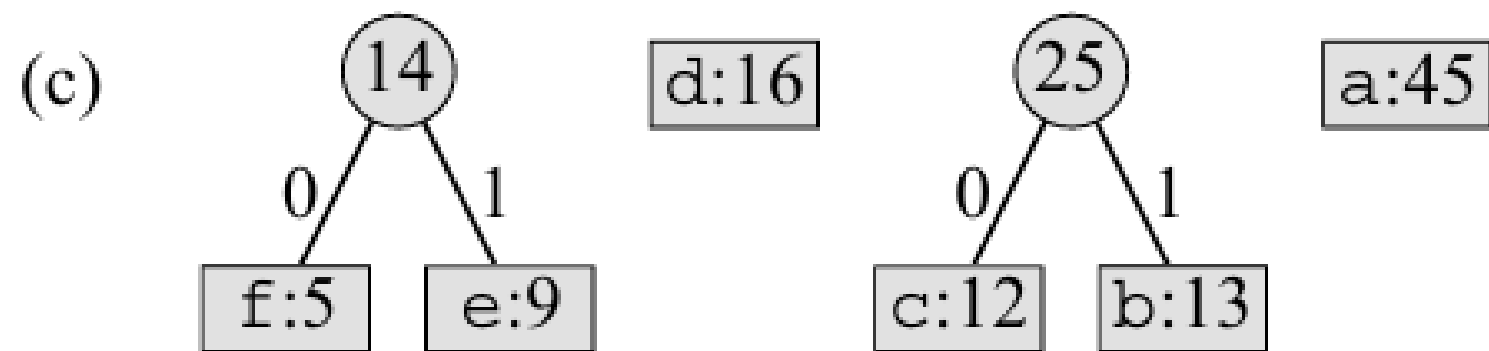
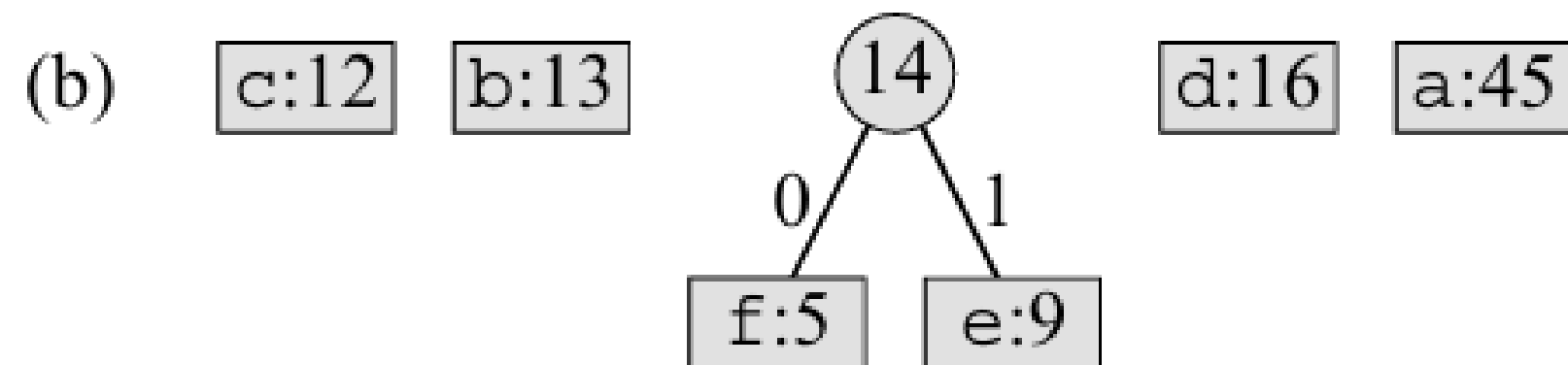


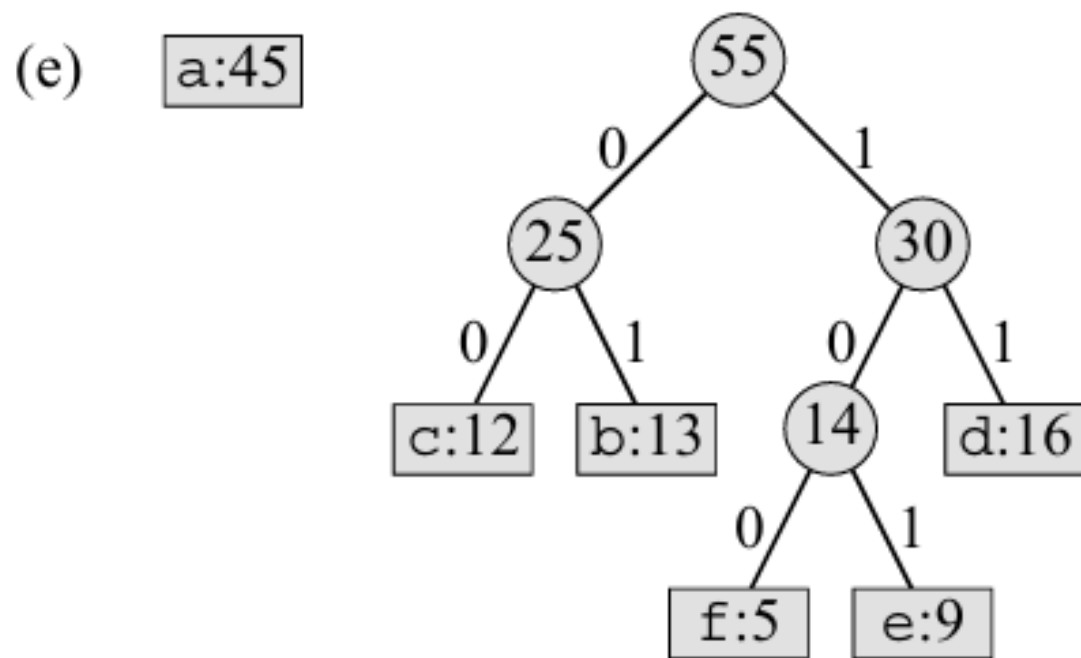
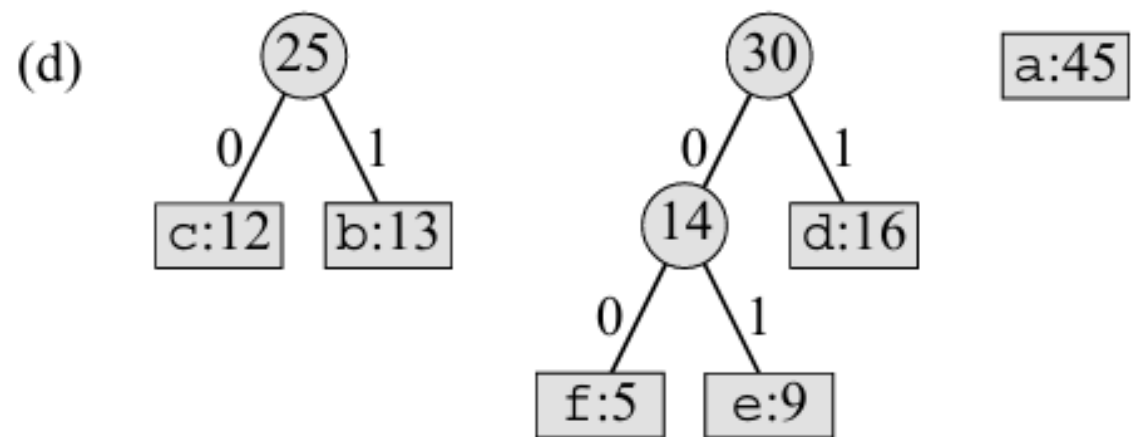
=> Leads to Ambiguity

Huffman code - Algorithm

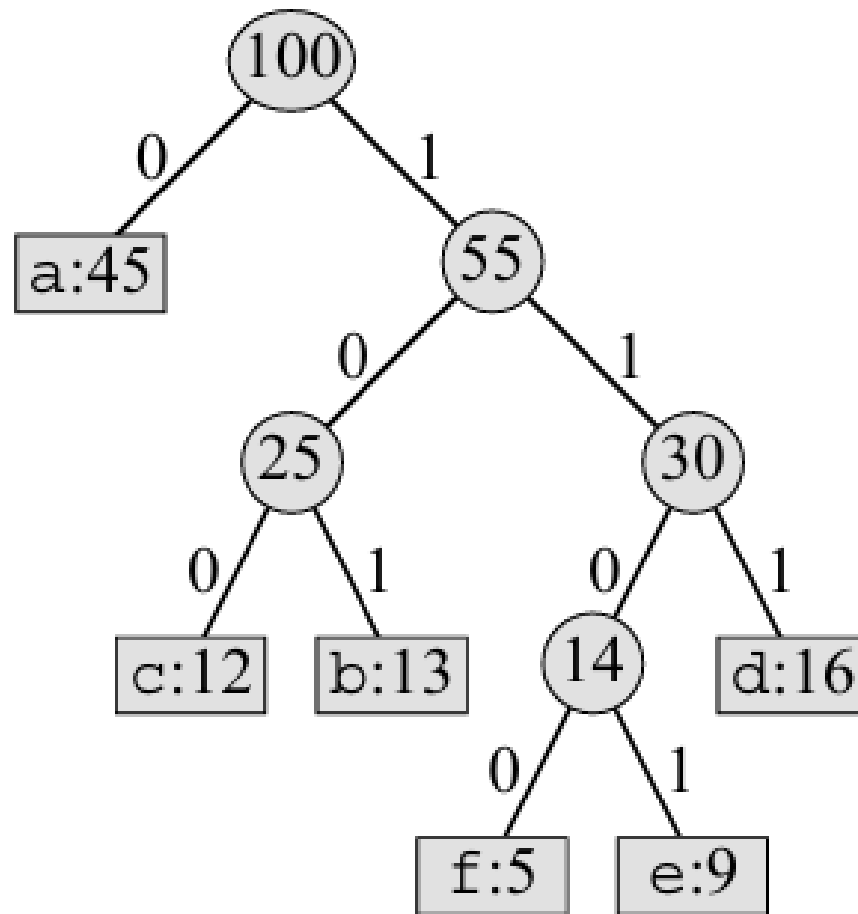
```
Huffman (C) // C- Set of n characters {a,b,c,d,e, f}
{
    n = |C|
    Q = C // Min-Priority Queue
    for i = 1 to n - 1
    {
        z = allocate new node
        z.left = x = Extract-Min(Q) // Use Min heap
        z.right = y = Extract-Min(Q) // Use Min heap
        z.freq = x.freq + y.freq
        INSERT(Q,z) // Inserting node in linked list
    }
    return Extract-Min(Q) // return the root of the tree
}
```

(a) f:5 e:9 c:12 b:13 d:16 a:45





(f)



Time complexity of Huffman code

Building Huffman Tree

- **Operation:** Insert all n symbols into a min-priority queue, then repeatedly extract the two smallest and insert their combined node.
- **Complexity:**
 - Initial heap build: $O(n)$
 - Each of $n - 1$ iterations: two EXTRACT-MIN operations and one INSERT $\rightarrow$ each $O(\log n)$
 - Total: $O(n \log n)$